



TonmaiD Vegetation Management – Administrator Manual



October 2024

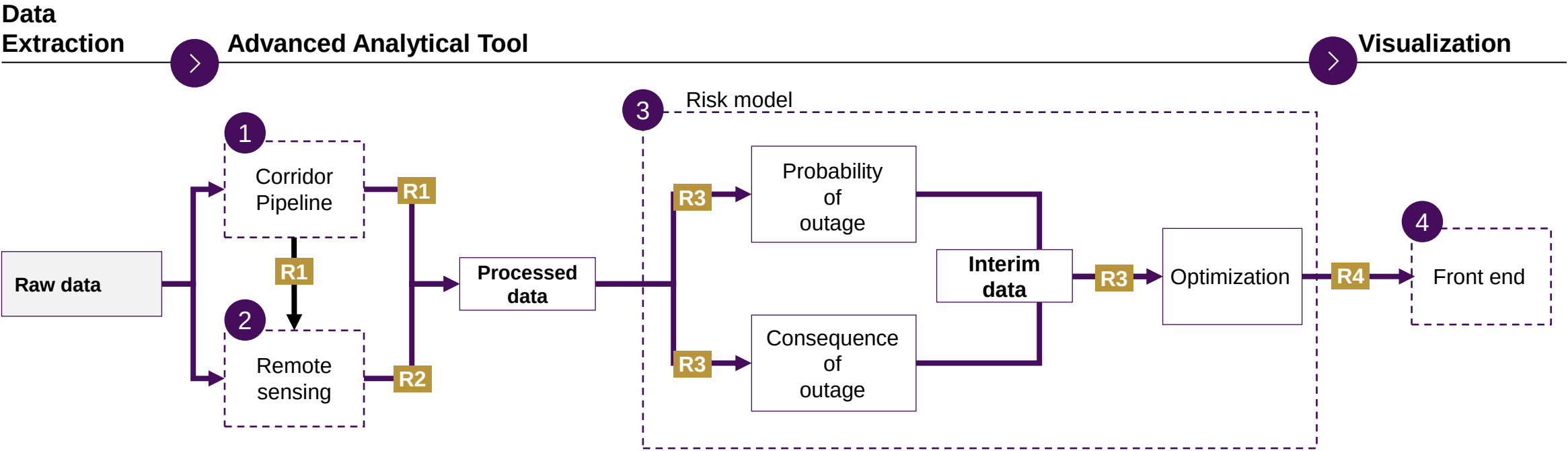
Contents

Overview

1. Corridor model
2. Remote sensing
3. Risk model
4. Front-end

Vegetation pipeline schema

Detailed next



Repository (R)

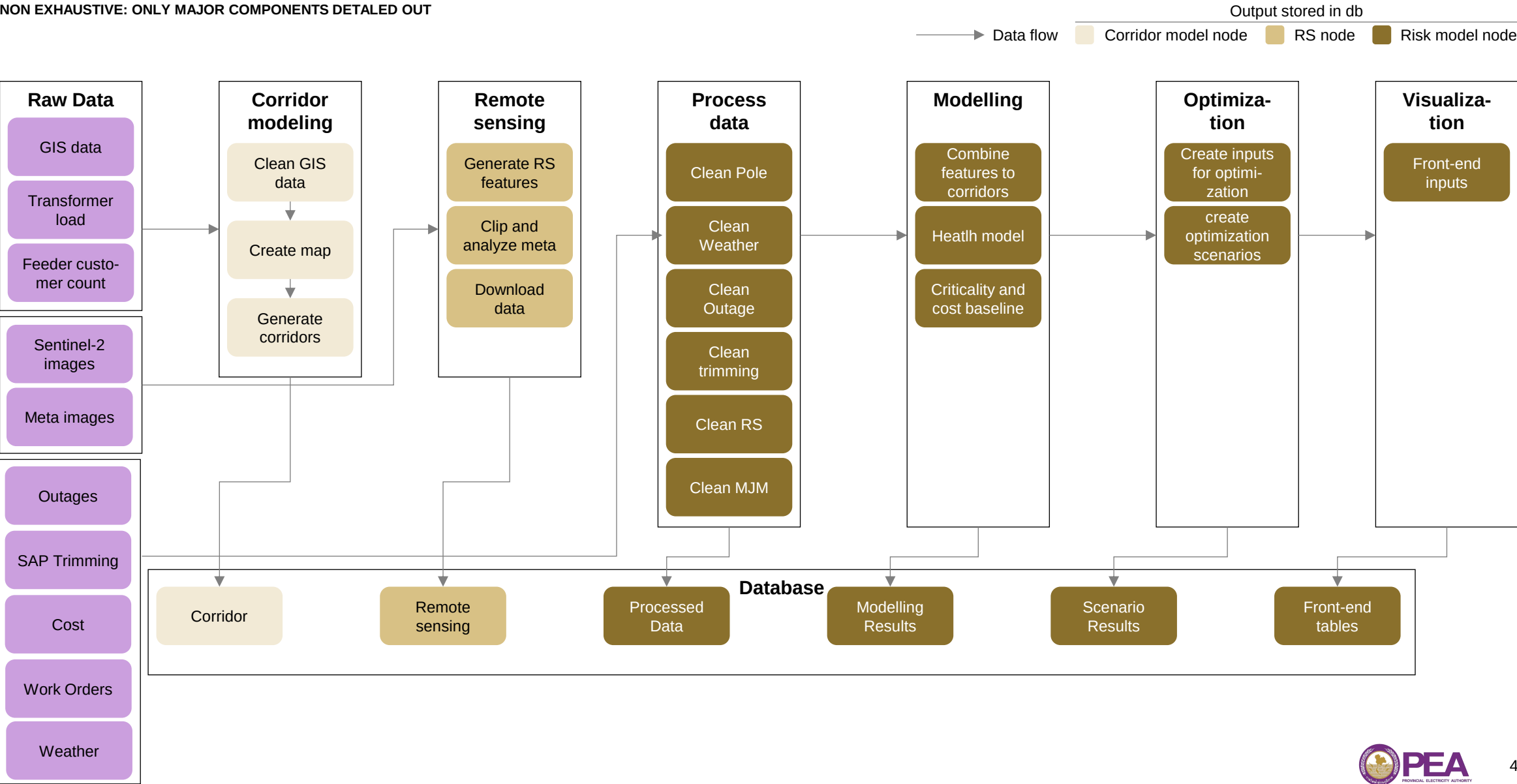
- R1: Corridor model
- R2: Remote sensing
- R3: Risk model
- R4: Front End

Legend

Git repo Data Flow /Code

Data Pipelines access database and local folder to generate visuals and scenario results

NON EXHAUSTIVE: ONLY MAJOR COMPONENTS DETAILED OUT



Contents

Overview

1. Corridor model

Data structure

Code repository

2. Remote sensing

3. Risk model

4. Front-end

Contents

Overview

1. Corridor model

– Data structure

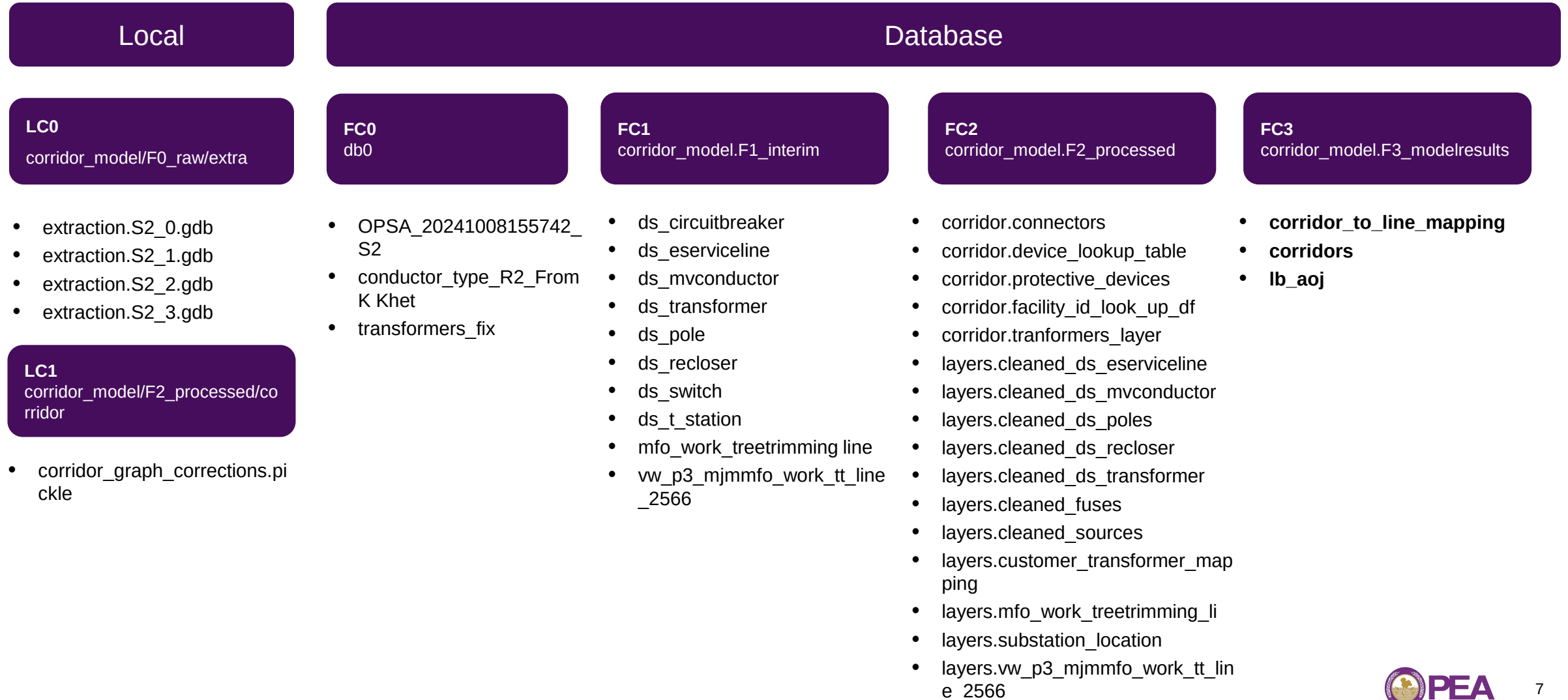
Code repository

2. Remote sensing

3. Risk model

4. Front-end

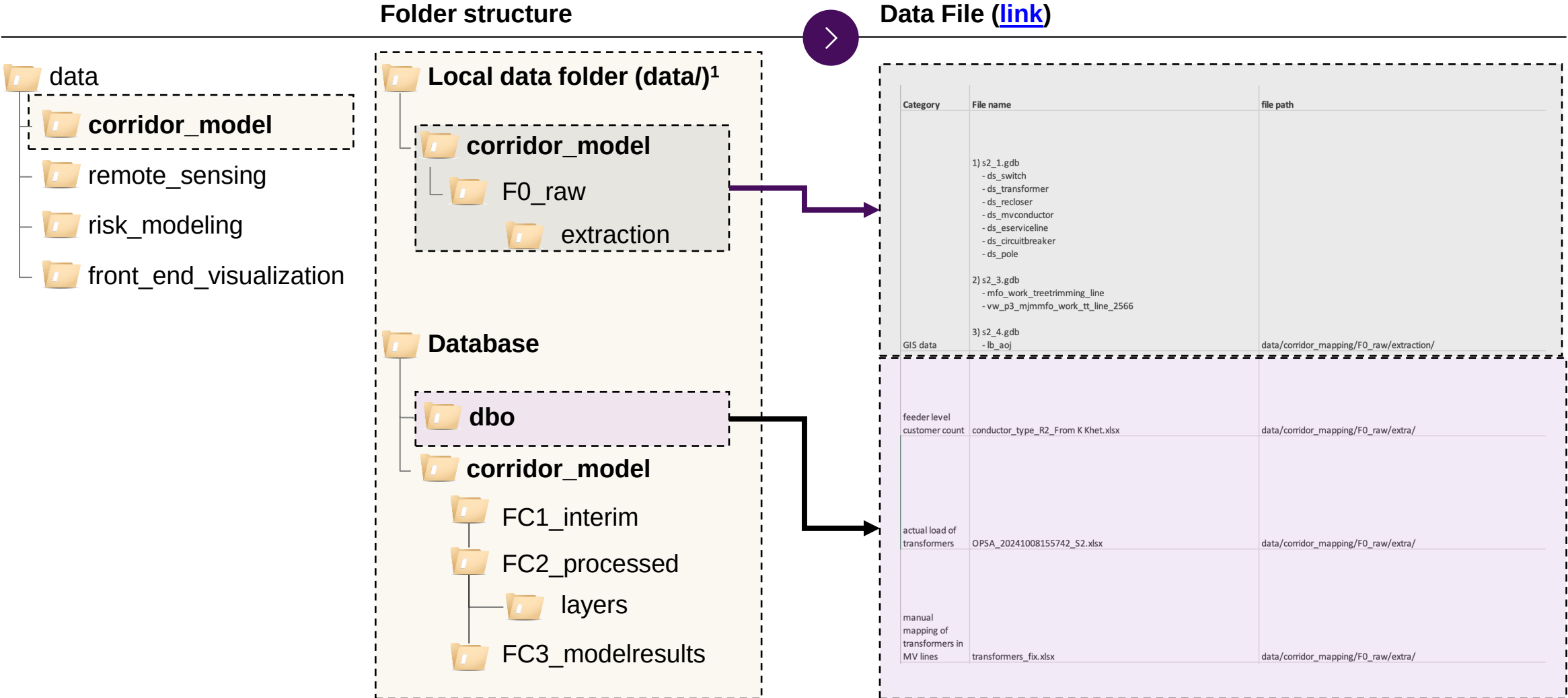
1 Corridor model – Data structure



1 Corridor model - Data structure

 Raw files to be updated in Local data folder (data/)¹  Raw files to be updated in database

Raw files required and to be updated to run detailed in Data File ([link](#))

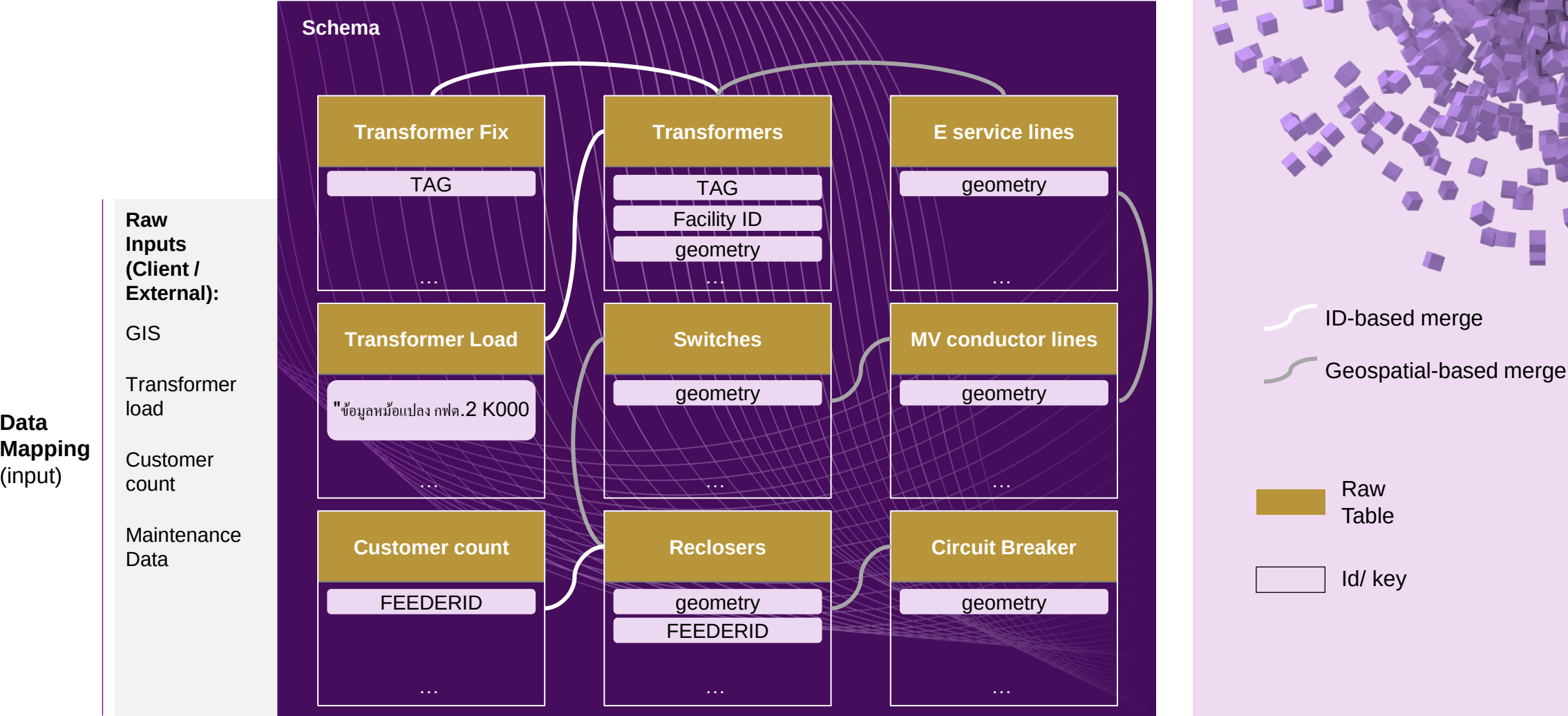


1. The location of the local "data/" folder is a parameter in the code determined by "base path". If this folder wants to be located in a sharepoint or any other location, it is enough with adapting "base path" parameter in the code to the desired one

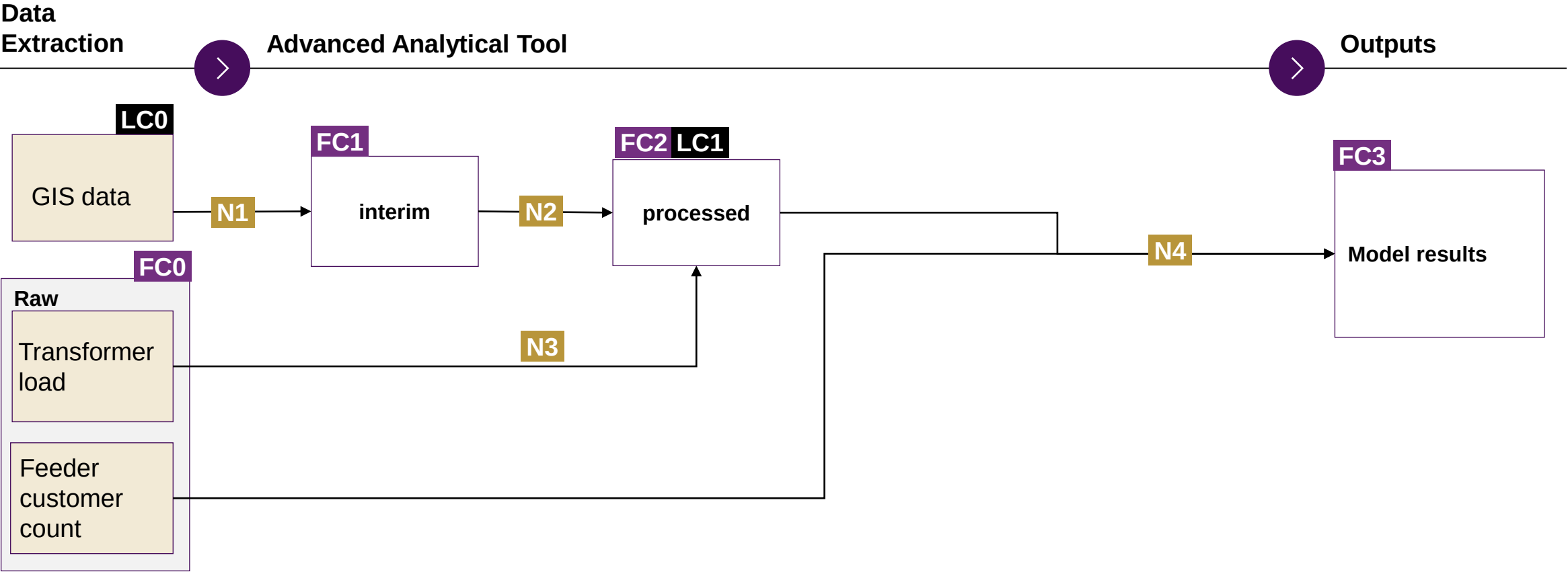
1 Corridor Model - ERD

Entity Relationship Diagram (ERD) for Corridor Model Pipeline input tables

NON-EXHAUSTIVE: ONLY MAJOR COMPONENTS DETAILED-OUT



1 Corridor model - Schema



Notebooks (N)	Data Folders (F)	Legend
N1: 01_download_data	FC0: dbo	PEA data
N2: 02_process_data	LC0: corridor_model/F0_raw/extra	notebooks
N3: 03_create_network_graph	FC1: corridor_model.F1_interim	Fi Database path
N4: 04_current_state_corridor_tracing	FC2: corridor_model.F2_processed	Li Local path
	LC1: corridor_model/F2_processed/corridor	
	FC3: corridor_model.F3_modelresults	

Contents

Overview

1. Corridor model

Data structure

– Code repository

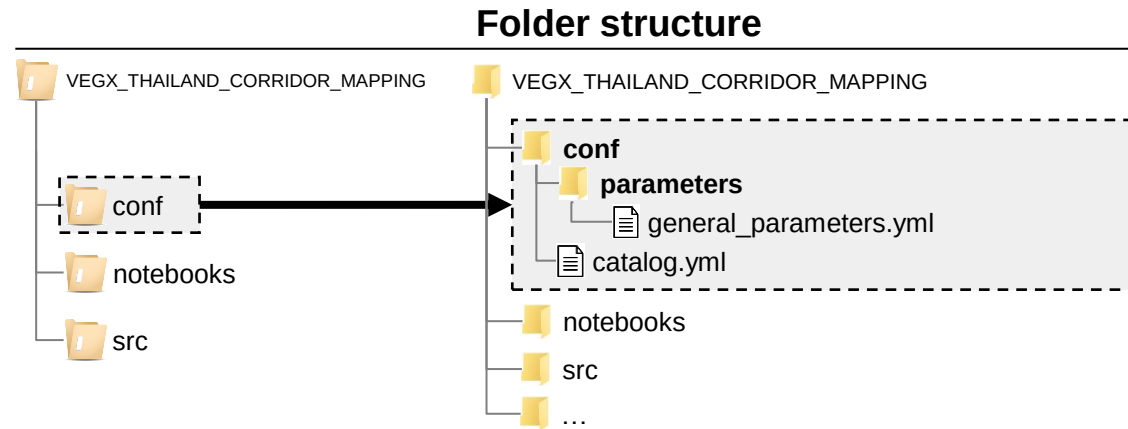
2. Remote sensing

3. Risk model

4. Front-end

1 Corridor model – conf/

conf/ - Catalog of files and parameters



> Content

conf

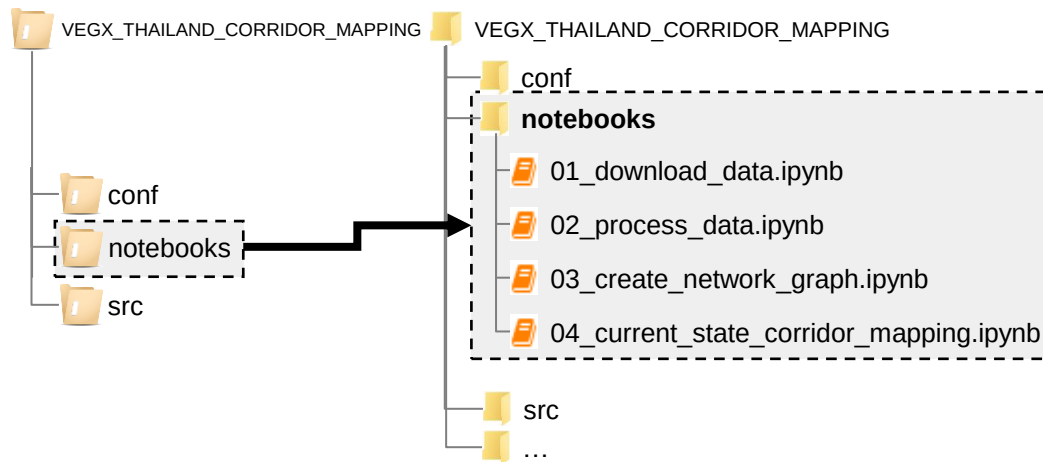
- **parameters¹**: folder for yml file on parameters in each repository
 - **general_parameters.yml**: it contains the list of variables used throughout each repository
- **catalog.yml**: contains the general files of the project.

1. The files contained in the catalog/ and parameters folders can have any nomenclature, but it is good practice to give them an identifying name (folders for catalog, pipelines for parameters).

1 Corridor model - Notebooks

notebooks/ - Examples and code repository

Folder structure



> Content (read.me [link](#))

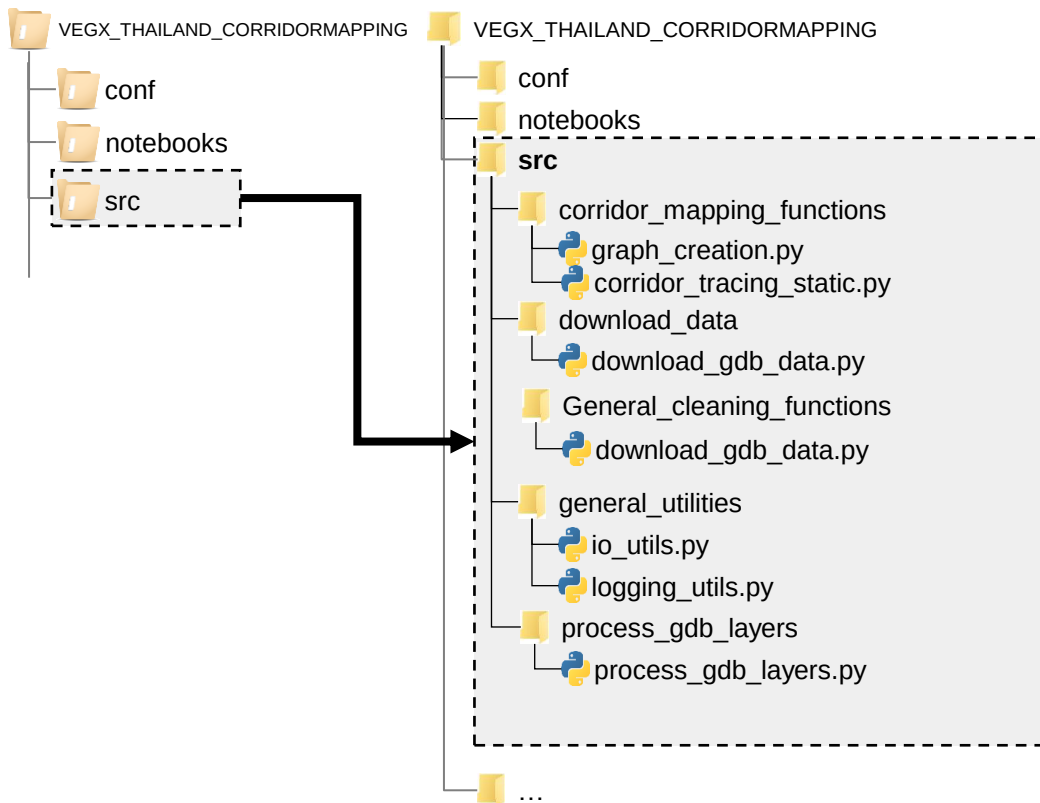
notebooks/

- **01_download_data.ipynb**: downloads relevant geospatial layers to map corridors
- **02_process_data.ipynb**: Filters geospatial layers to relevant col
- **03_create_network_graph.ipynb**: creates network graph between devices and overhead lines
- **04_current_state_corridor_mapping.ipynb**: create corridor mapping outputs

1 Corridor model - Source scripts

notebooks/ - Examples and code repository

Folder structure



Content

src

- **graph_creation.py**: Functions to create create corridor mapping
- **corridor_tracing_static.py**: functions to create corridor level output and line to corridor level outputs
- **download_gdb_data.py**: Function for parallelized downloading of gis layers
- **io_utils.py**: functions to import and save data files
- **logging_utils.py**: logging functions used throughout repository pipeline
- **Process_gdb_layers.py**: prepares specific layers to be inputted into the corridor model pipeline

Contents

Overview

1. Corridor model

2. Remote sensing

Data structure

Code structure

3. Risk model

4. Front-end

Contents

Overview

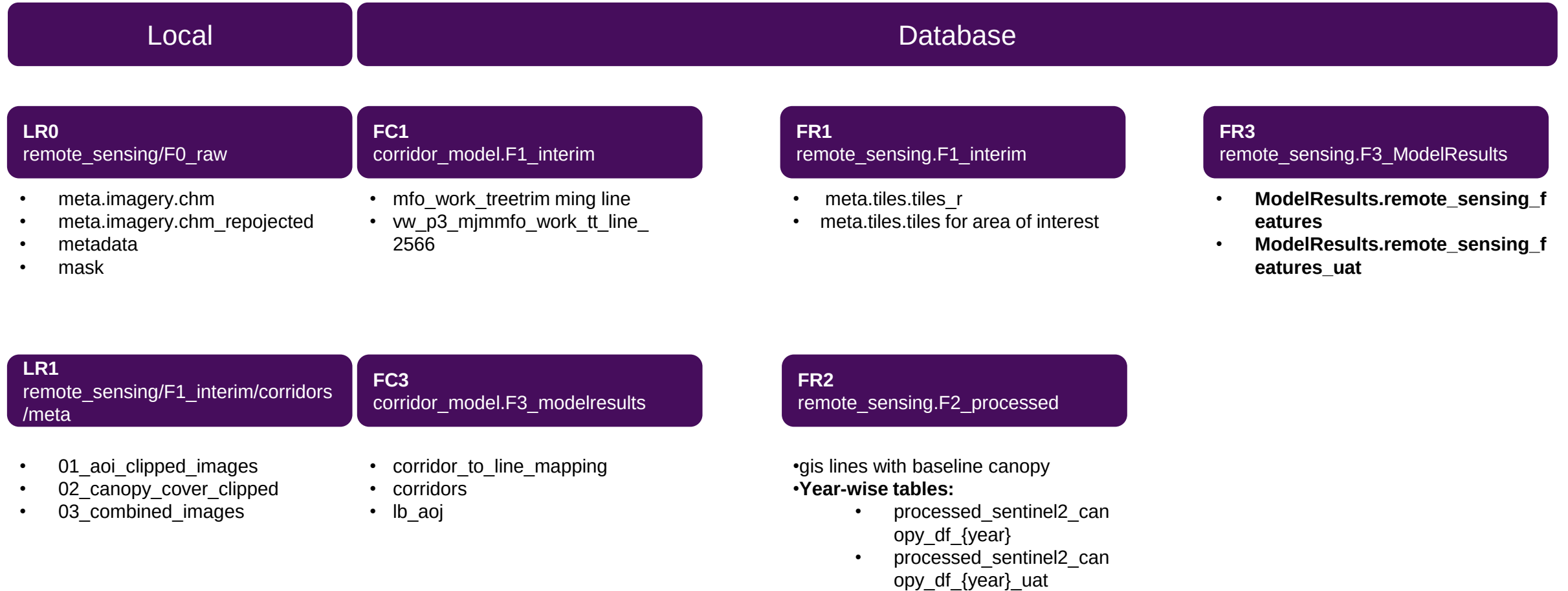
1. Corridor model
2. Remote sensing

– Data structure

Code structure

3. Risk model
4. Front-end

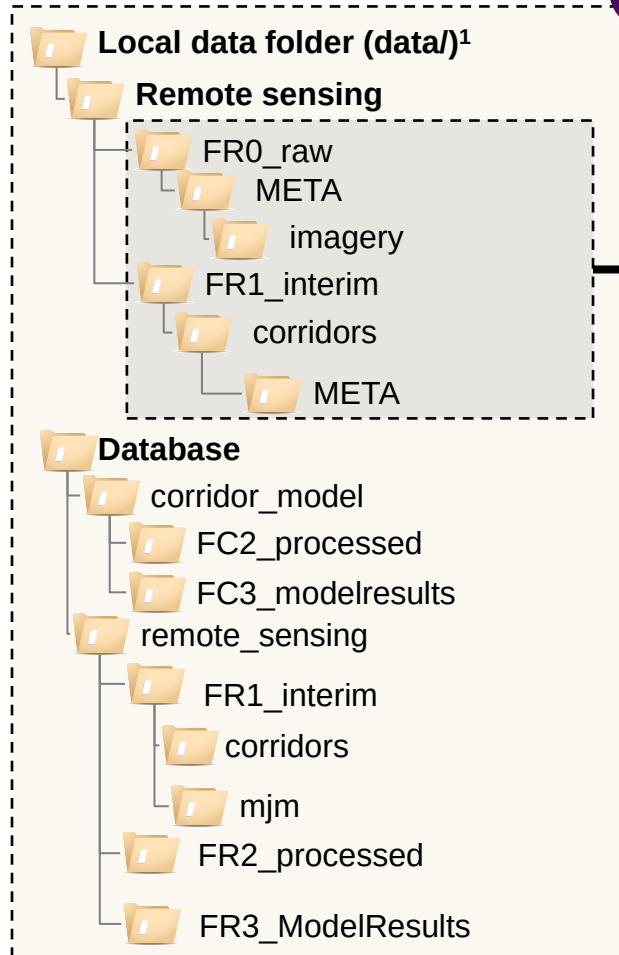
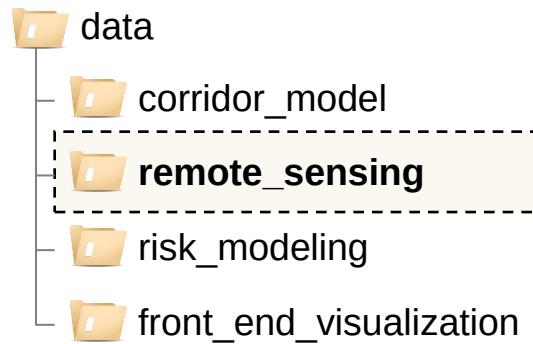
2 Remote sensing– Data structure



2 Remote sensing - Data structure

[] Raw files to be updated in Local data folder (data/)¹

Folder structure



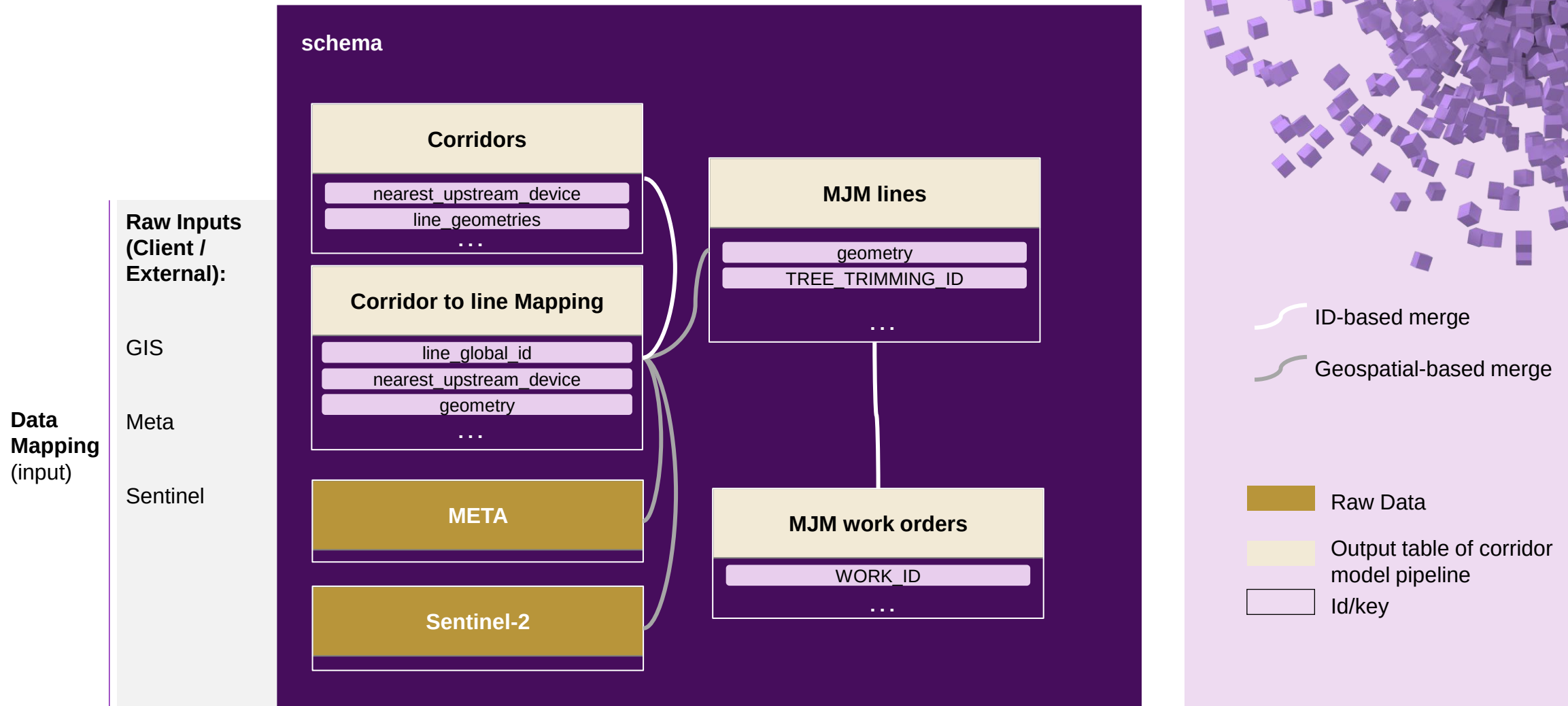
Raw files required and to be updated to run detailed in Data File ([link](#))

A	B	D
Category	Folder/File name	Folder Path
META 1m canopy detection	META	data/remote_sensing/FO_raw/meta/
Sentinel-2	sentinel-2	data/remote_sensing/F2_processed/

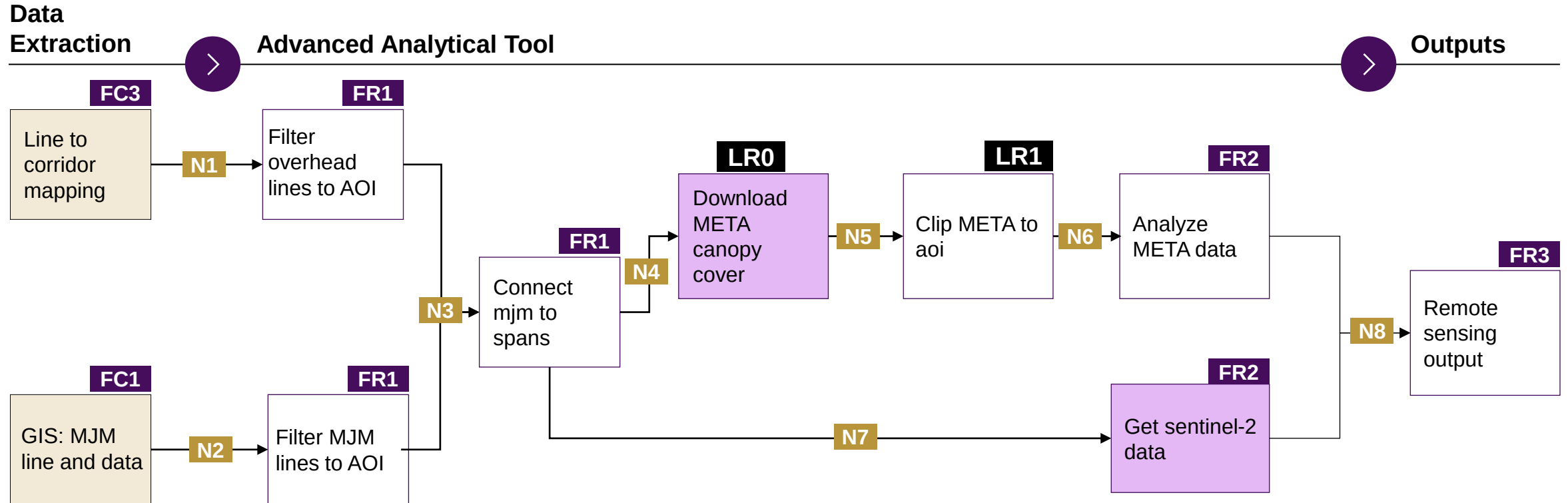
1. The location of the local "data/" folder is a parameter in the code determined by "base path". If this folder wants to be located in a sharepoint or any other location, it is enough with adapting "base path" parameter in the code to the desired one

2 Remote Sensing - ERD

Entity Relationship Diagram (ERD) for Remote Sensing pipeline input tables



2 Remote sensing - Schema



Notebooks (N)

N1: 01_filter_corridors_to_aoi
 N2: 02b_mjm_filter_to_aoi
 N3: 02c_connect_mjm_to_spans
 N4: 03_download_meta_canopy_cover
 N5: 04_clip_meta_to_aoi
 N6: 05_analyze_meta_data
 N7: 06_get_sentinel2_data
 N8: 07_remote_sensing_outputs

Data Folders (F)

L0: remote_sensing/F0_raw
 L1: remote_sensing/F1_interim/corridors/meta
 FC1: corridor_model.F1_interim
 FC3: corridor_model.F3_modelresults
 FR1: remote_sensing.F1_interim
 FR2: remote_sensing.F2_processed
 FR3: remote_sensing.F3_ModelResults

Legend

External data
 Corridor model pipeline
 Remote sensing output path
 Local path
 notebooks

Contents

Overview

1. Corridor model
2. Remote sensing

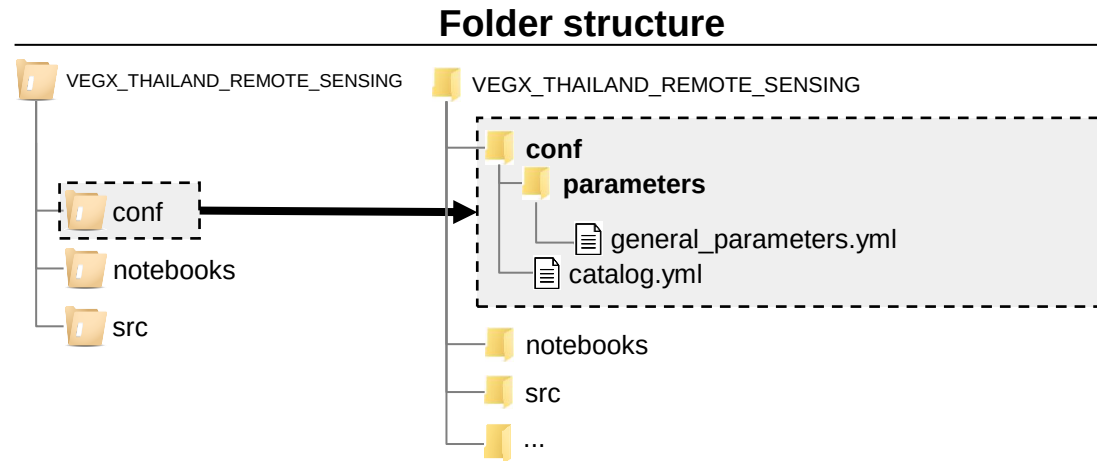
Data structure

– Code structure

3. Risk model
4. Front-end

2 Remote sensing model – conf/

conf/ - Catalog of files and parameters



> Content

conf

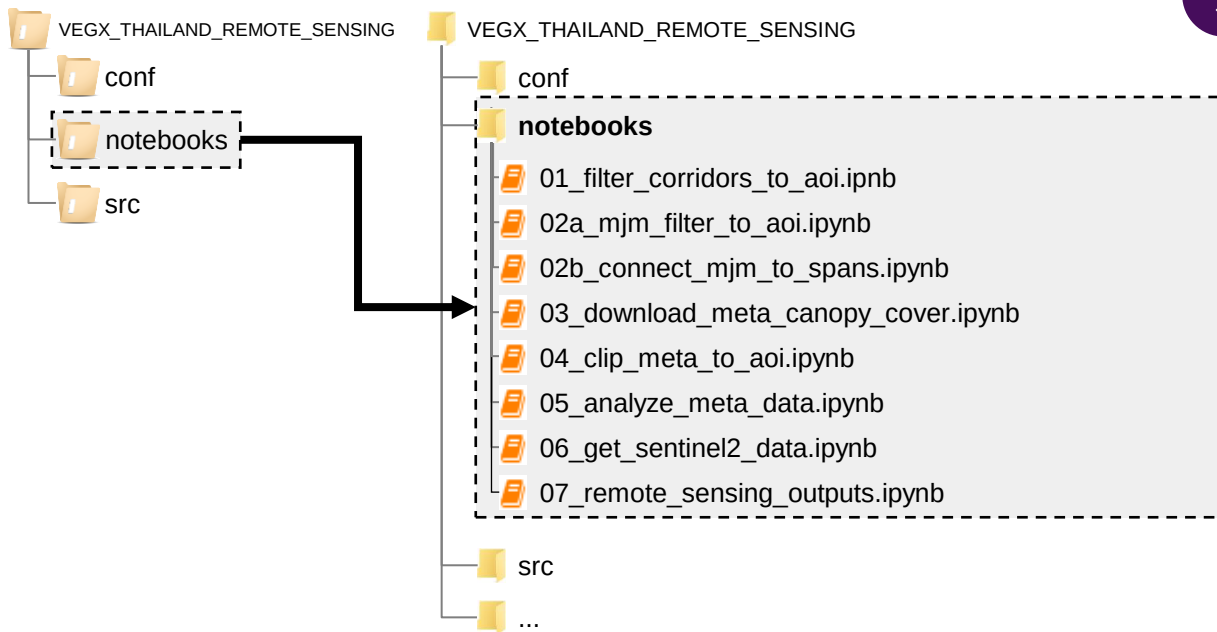
- **parameters¹**: folder for yml file on parameters in each repository
 - **general_parameters.yml**: contains parameters to consider for remote sensing
- **catalog.yml**: contains the general files of the project.

1. The files contained in the catalog/ and parameters folders can have any nomenclature, but it is good practice to give them an identifying name (folders for catalog, pipelines for parameters).

2 Remote sensing - Notebooks

notebooks/ - Examples and code repository

Folder structure



Content (read.me [link](#))

notebooks/

- **01_filter_corridors_to_aoi.ipynb**: filters corridors to areas of interest
- **02a_mjm_filter_to_aoi.ipynb**: filter mjm data to corridors area of interest
- **02b_connect_mjm_to_spans.ipynb**: connect mjm data to spans
- **03_download_meta_canopy_cover.ipynb**: download meta canopy tree cover data
- **04_clip_meta_to_aoi.ipynb**: clip meta data to area of interest
- **05_analyze_meta_data.ipynb**: extract canopy features
- **06_get_sentinel2_data.ipynb**: download sentinel 2 data to area of interest
- **07_remote_sensing_outputs.ipynb**: Calculate canopy density values for meta and sentinel2 combination

2 Remote sensing source scripts

src/ - Examples and code repository

Folder structure



Content

src/

- **clipping_data.py**: script for filtering geospatial data to areas of interest
- **corridor_cleaning.py**: functions to subset corridors to areas of interest
- **baseline_density.py**: functions to calculate base line tree canopy density values
- **Sentinel2_density.py**: functions to combine density calculations for baseline and sentinel 2
- **Combine_baseline_and_sentinel2.py**: functions to create baseline density change through META and Sentinel-2
- **mjm_processing.py**: functions to download and process mjm data
- **io_utils.py**: functions to import and save data files
- **logging_utils.py**: logging functions used throughout repository pipeline

Contents

Overview

1. Corridor model
2. Remote sensing

3. Risk model

Data structure

Code repository

4. Front-end

Contents

Overview

1. Corridor model
2. Remote sensing
3. Risk model

– Data structure

Code repository

4. Front-end

3 Risk model - data structure

Database

FA0 dbo

- 2021_tree_WO
- 2022_tree_WO
- 2023_tree_WO
- Rate_card_South S2
- sap_manual
- S2
- 2023-12-01-Report No.52 Year to Date PEA-ALL_applied filter for Asset management.xlsx¹
- 2022-12-01-Report No.52 Year to Date PEA-ALL.xlsx¹
- 2022-01-01-รายงานที่ 52 รายปี PEA-ALL.xlsx¹
- 2021-01-01-รายงานที่ 52 รายปี PEA-ALL.xlsx¹
- ESN 2024.xlsx¹

FC3

corridor_model.F3_modelresults

- corridor_to_line_mapping
- corridors

FA3

risk_modeling.F3_modelingResults

- model_output_decile_image
- model_output_decile_km_image
- model_outputs_feature_importance
- model_outputs_health_concentration_importance
- model_outputs_shaps_and_values
- model_outputs_outage_chance_scores
- model_outputs_roc_image
- model_outputs_saifi_calibration
- model_outputs_shaps_image
- ...

FR3

remote_sensing.F3_ModelResults

- remote_sensing_features

FA4

risk_modeling.F4_optimizationParameters

- density_line
- inefficiency_factor_df
- model_outputs_criticality_concentration_image
- outputs_criticality_score

FA7

risk_modeling.F7_front_end_inputs

- front_end_input_features
- model_shap_features
- optimization_metrics
- optimization_scenarios

FA1

risk_modeling.F1_processed

- outages.features_outages
- trimming.features_trimming_point
- weather.features_weather
- remote_sensing.remote_sensing_corridor_features
- poles.cleaned_poles
- ...

FA5

risk_modeling.F5_optimizationInputs

- agg_scores_and_criticality_corridor
- risk_concentration_image

FA2

risk_modeling.F2_masterTable

- model_inputs_prediction_set
- model_inputs_training_set
- model_inputs_testing_set

FA6

risk_modeling.F6_optimizationResults

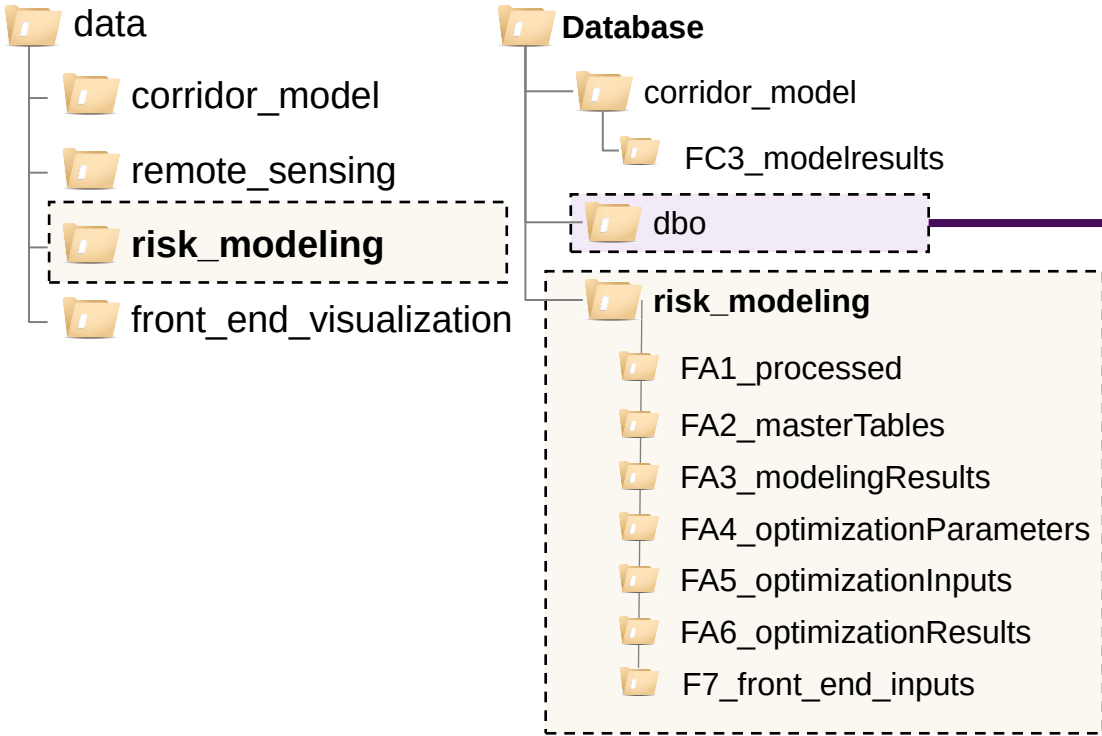
- curve_risk_image
- opti_curve_saifi_image
- metrics_scenario_1
- metrics_scenario_2
- metrics_chosen_scenario
- scenario_1
- scenario_2
- scenario_chosen
- scenario_budget

1. Pending upload to database

3 Risk model - data structure

 Raw files to be updated in database

Folder structure



Raw files required and to be updated to run detailed in Data File([link](#))

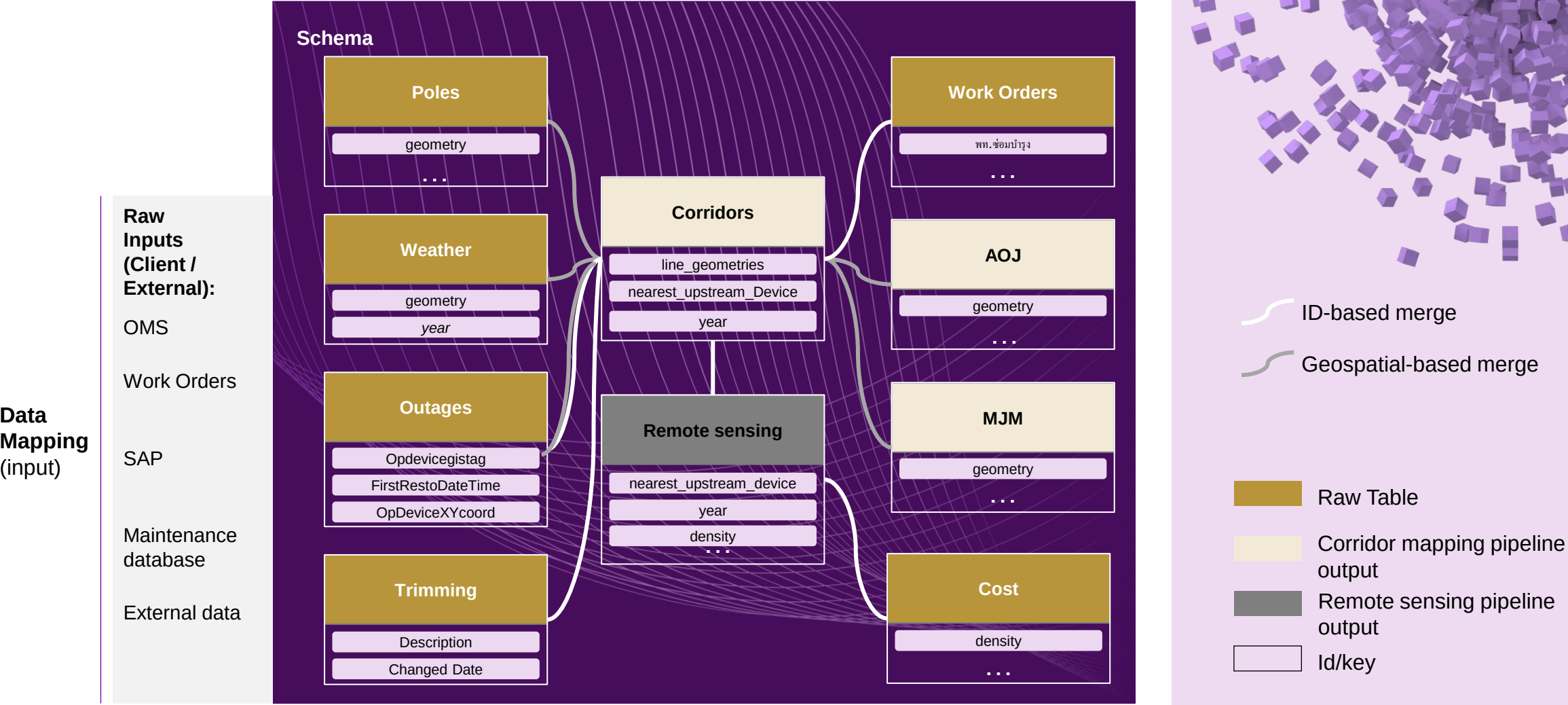
Category	File name	file path
outage history	1) 2023-12-01-Report No.52 Year to Date PEA-ALL_applied filter for Asset management.xlsx	data/risk_modeling/F0_raw/outages/
	2) 2022-12-01-Report No.52 Year to Date PEA-ALL.xlsx	
	3) 2022-01-01-รายงานที่ 52 รายปี PEA-ALL.xlsx	
	4) 2021-01-01-รายงานที่ 52 รายปี PEA-ALL.xlsx	
	5) ESN 2024.xlsx	
SAP trimming data	1) S2.xlsx 2) sap_manual.xlsx	data/risk_modeling/F0_raw/trimming/
work order data	1) 2021 เฉพาะต้นไม้ + W.O..xlsx 2) 2022 เฉพาะต้นไม้ + W.O..xlsx 3) 2023 เฉพาะต้นไม้ + W.O..xlsx	data/risk_modeling/F0_raw/wo/

1. The location of the local "data/" folder is a parameter in the code determined by "base path". If this folder wants to be located in a sharepoint or any other location, it is enough with adapting "base path" parameter in the code to the desired one

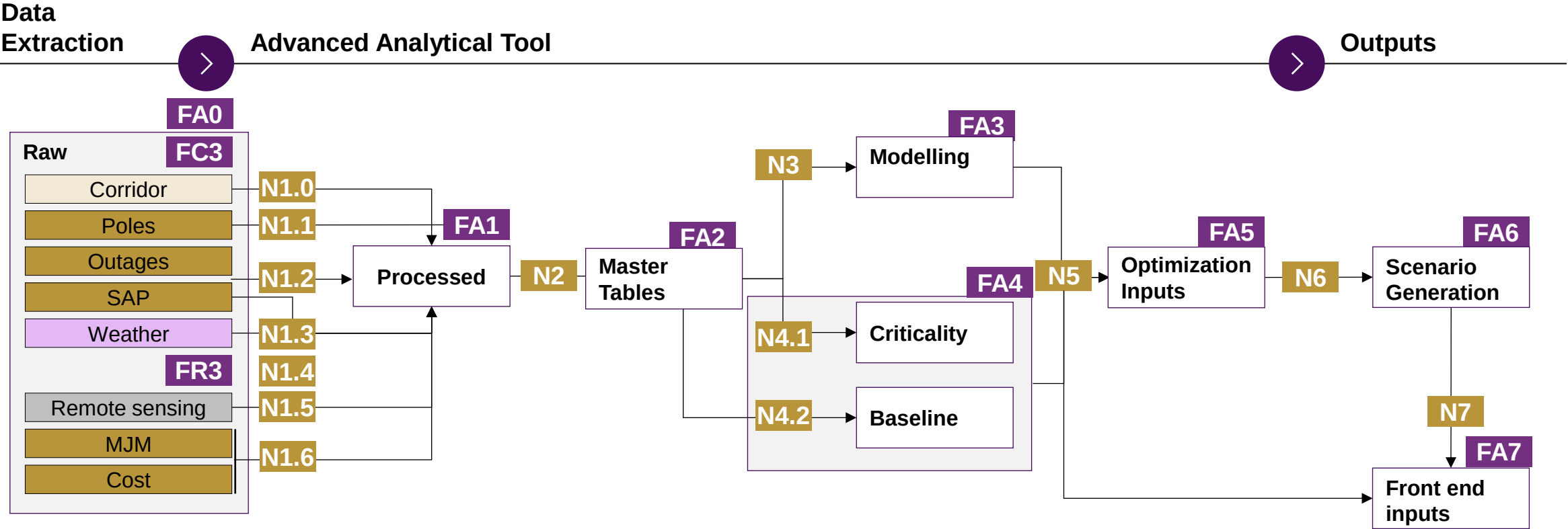
3 Risk model - ERD

Entity Relationship Diagram (ERD) for Risk Model pipeline input tables

NON-EXHAUSTIVE: ONLY MAJOR COMPONENTS DETAILED-OUT



3 Risk model - schema



Notebooks (N)

N1.0: 1.0_corridor_cleaning.ipynb	N2: 2_create_model_inputs.ipynb
N1.1: 1.1_pole_cleaning.ipynb	N3: 3_outage_probability_model.ipynb
N1.2: 1.2_weather_cleaning.ipynb	N4.2: 4.1_criticality_model.ipynb
N1.3: 1.3_outage_cleaning.ipynb	N4.1: 4.2_cost_baseline.ipynb
N1.4: 1.4_trimming_cleaning	N5: 5_optimization_inputs.ipynb
N1.5: 1.5_remote_sensing_cleaning.ipynb	N6: 6_scenario_generation.ipynb
N1.6: 1.6_mjm_cleaning.ipynb	N7: 7_outputs_for_front_end.ipynb

Data Folders (F)

FA0: dbo
FC3: corridor_model.F3_modelresults
FR3: remote_sensing.F3_ModelResults
FA1: risk_modeling.F1_processed
FA2: risk_modeling.F2_masterTable
FA3: risk_modeling.F3_modellingResults
FA4: risk_modeling.F4_optimizationParameters
FA5: risk_modeling.F5_optimizationInputs
FA6: risk_modeling.F6_optimizationResults
FA7: risk_modeling.F7_front_end_inputs

Legend

PEA data	notebooks
External data	Database path
Remote sensing	
pipeline output	
Corridor mapping	
pipeline output	

Contents

Overview

1. Corridor model
2. Remote sensing
3. Risk model

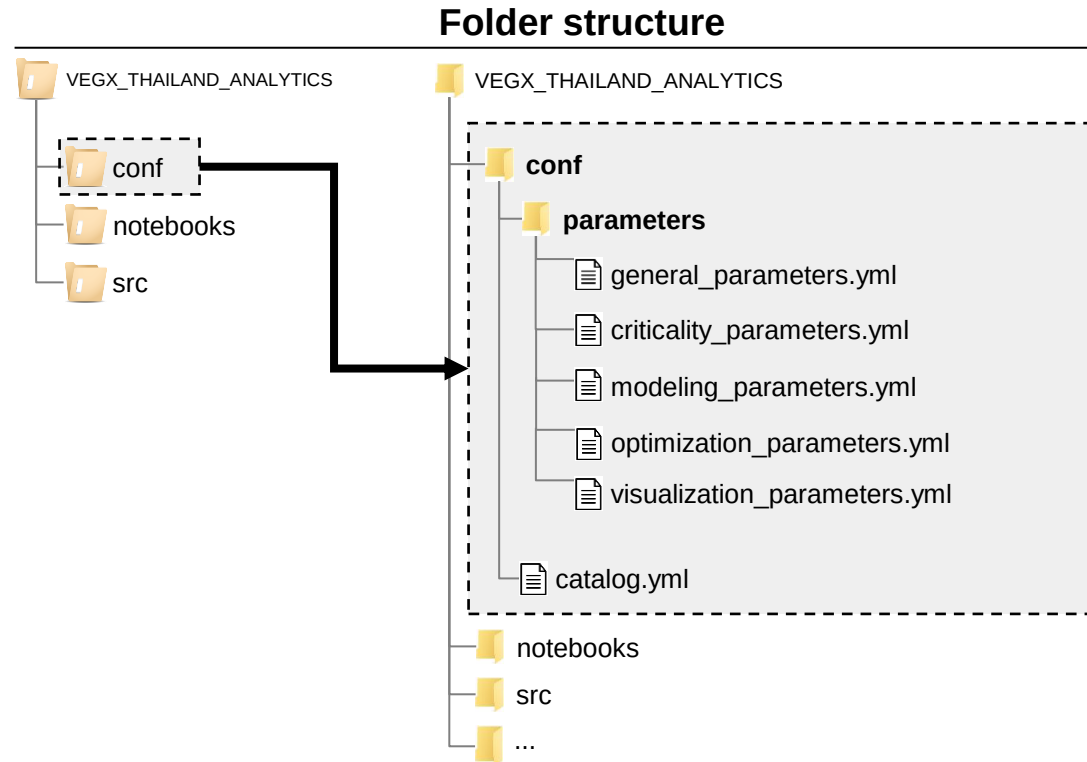
Data structure

– Code repository

4. Front-end

3 Risk model – conf/

conf/ - Catalog of files and parameters



> Content

conf

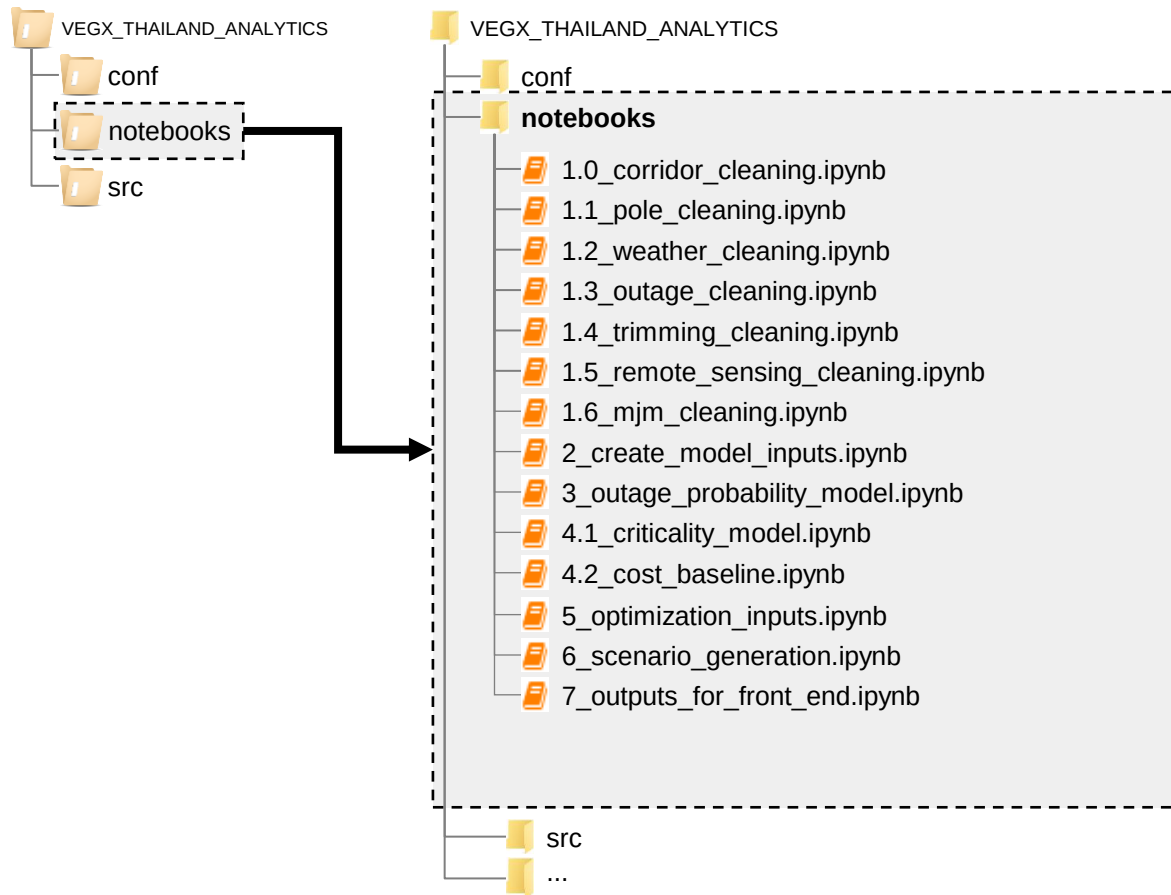
- **parameters¹**: folder for `yml` file on parameters in each repository
 - `general_parameters.yml`: parameters used throughout pipeline
 - `criticality_parameters.yml`: criticality parameters
 - `modeling_parameters.yml`: modeling parameters
 - `optimization_parameters.yml`: optimization parameters
 - `visualization_parameters.yml`: visualization parameters
- **catalog.yml**: contains the general files of the project.

1. The files contained in the `catalog/` and `parameters` folders can have any nomenclature, but it is good practice to give them an identifying name (folders for `catalog`, pipelines for `parameters`).

3 Risk model - notebooks

notebooks/ - Examples and code repository

Folder structure



Content (read.me [link](#))

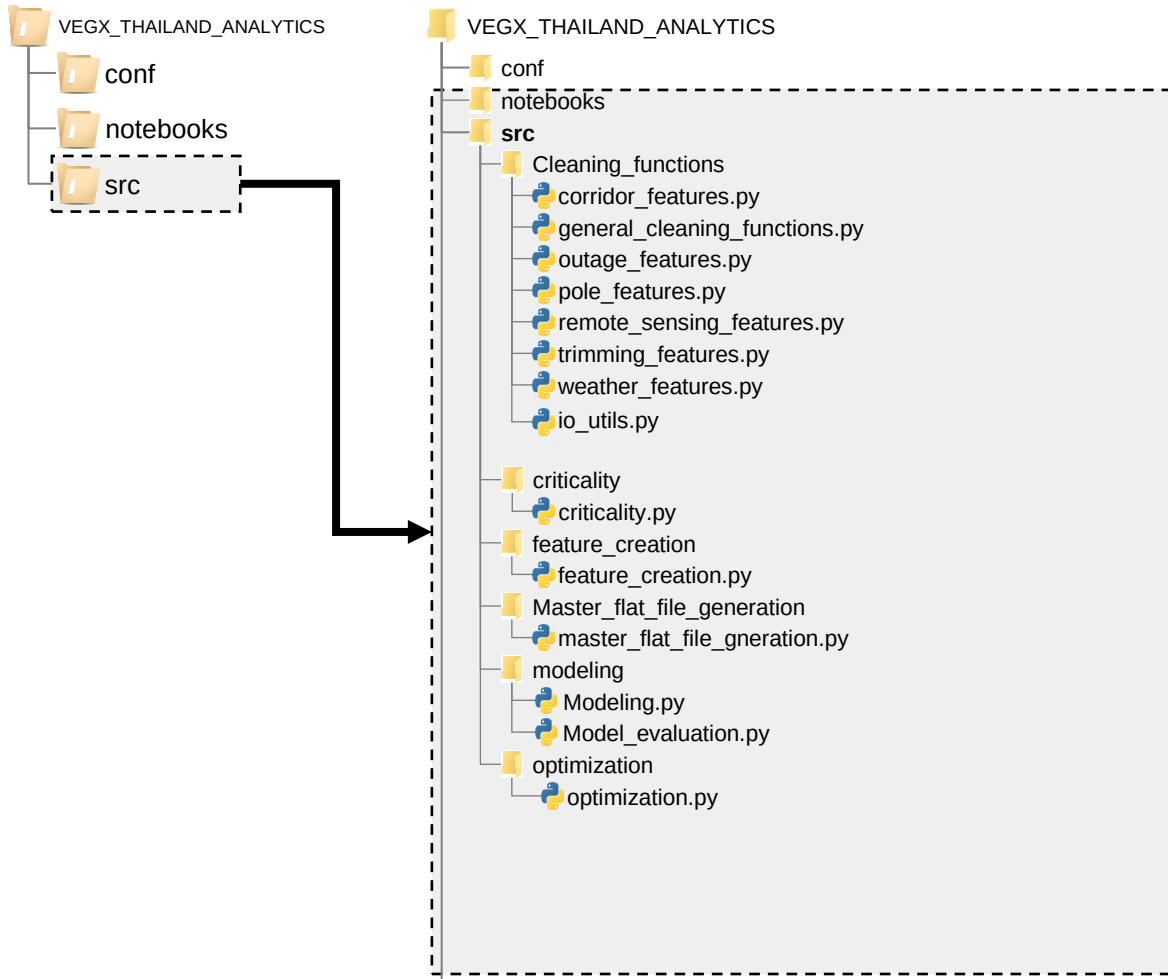
notebooks/

- **1.0_corridor_cleaning.ipynb**: filter corridors to areas of inters
- **1.1_pole_cleaning.ipynb**: create pole features
- **1.2_weather_cleaning.ipynb**: create weather features
- **1.3_outage_cleaning.ipynb**: create outage features
- **1.4_trimming_cleaning.ipynb**: create trimming features
- **1.5_remote_sensing_cleaning.ipynb**: create remote sensing features
- **1.6_mjm_cleaning.ipynb**: connect mjm lines to gis lines
- **2_create_model_inputs.ipynb**: combine features to corridors
- **3_outage_probability_model.ipynb**: create health model
- **4.1_criticality_model.ipynb**: create criticality model
- **4.2_cost_baseline.ipynb**: create baseline cost estimate
- **5_optimization_inputs.ipynb**: create inputs for optimizaiton
- **6_scenario_generation.ipynb**: create optimization scenarios
- **7_outputs_for_front_end**: create output dataframes for front end

3 Risk model - source scripts

notebooks/ - Examples and code repository

Folder structure



Content

src/

- **corridor_functions.py**: creates corridor features
- **general_cleaning_functions.py**: cleaning functions such as translations to english
- **outage_features.py**: creates outage features
- **pole_features.py**: Create pole features
- **remote_sensing_features.py**: maps remote sensing features
- **trimming_features.py**: processes trimming features
- **weather_features.py**: Creates and maps weather features
- **io_utils.py**: functions to import and save data files
- **criticality.py**: Criticality features
- **feature_creation.py**: Creates lagged features
- **Master_flat_file_generation.py**: combines all features for modeling
- **Modeling.py**: run model outage prediction
- **Model_evaluation.py**: evaluate model results
- **optimization.py**: optimization parameters

Contents

Overview

1. Corridor model
2. Remote sensing
3. Risk model

4. Front-end

Data Structure

Code repository

Contents

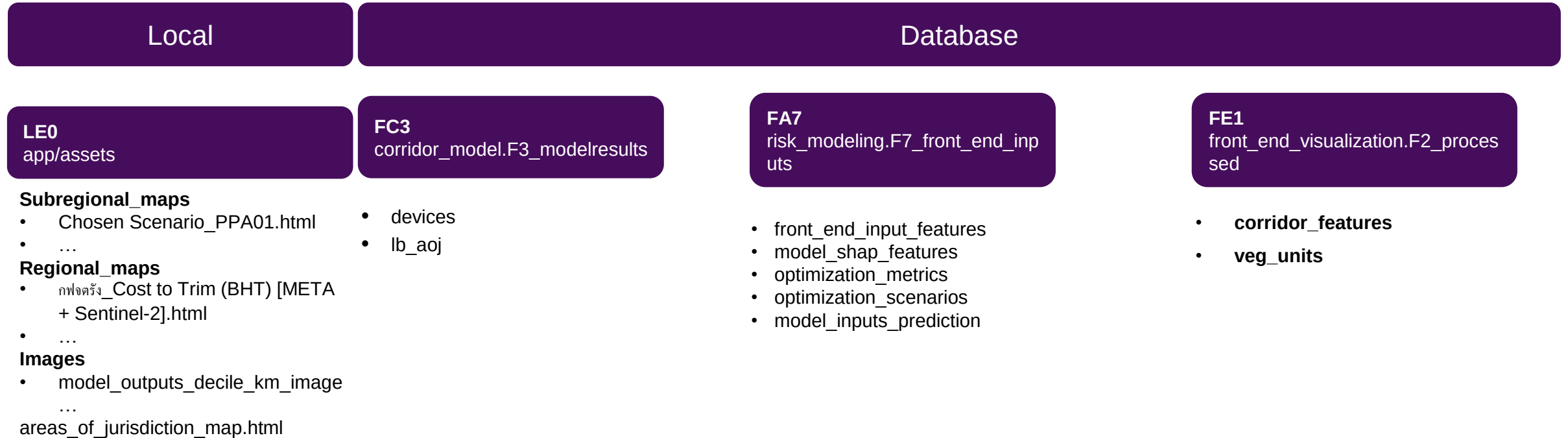
Overview

1. Corridor model
2. Remote sensing
3. Risk model
4. Front-end

– Data Structure

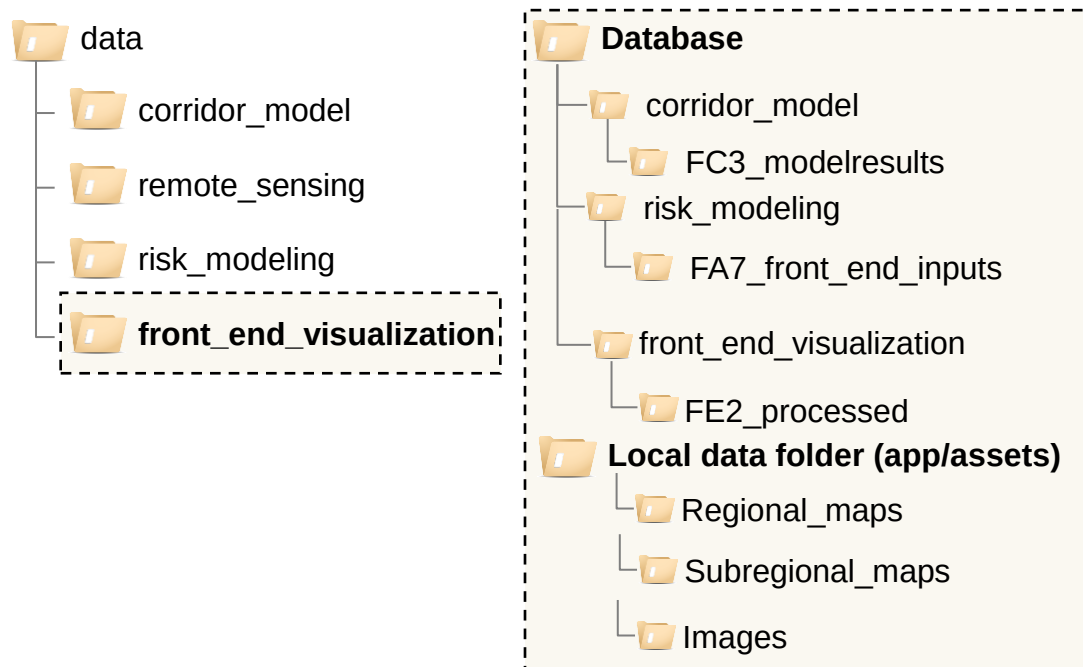
Code repository

4 Front-end - data structure



4 Front end - data structure

Folder structure



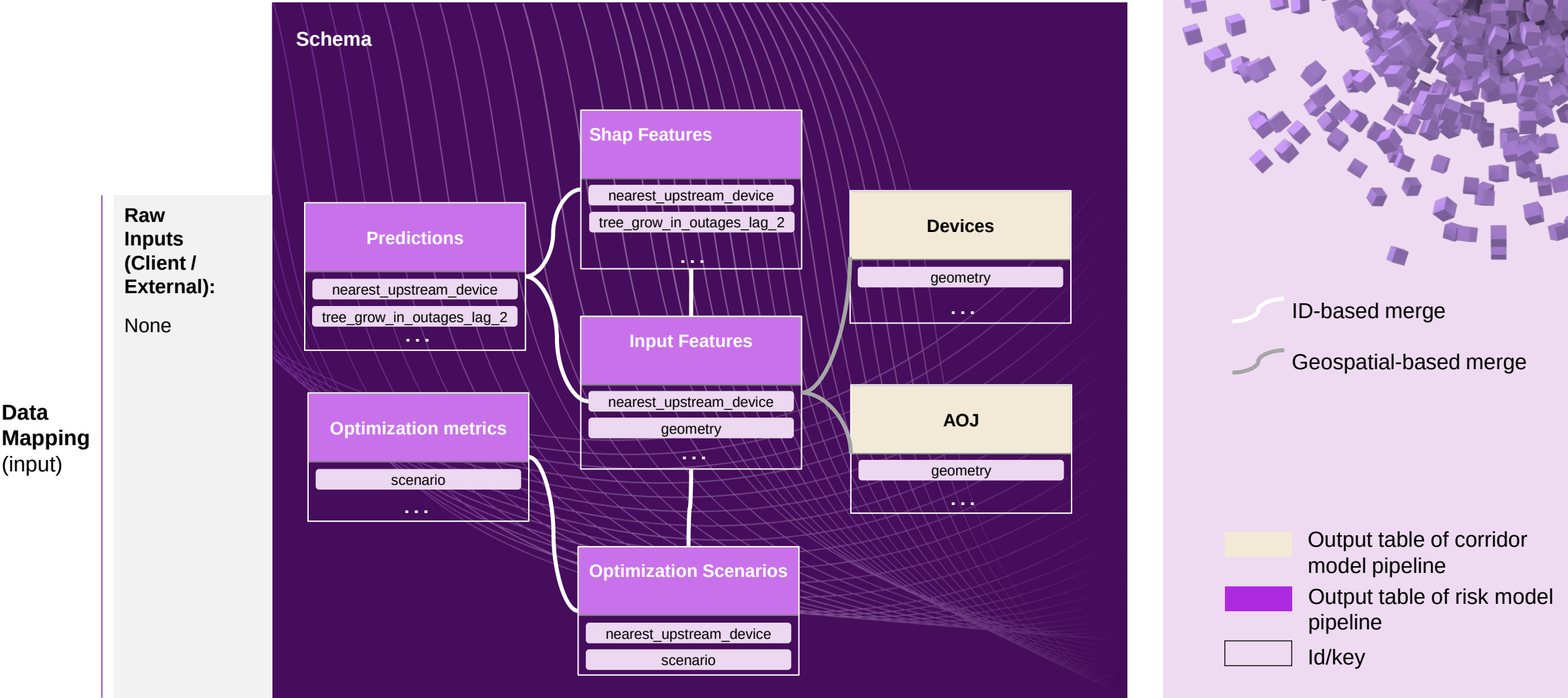
Front-end repo does not require of any file to be updated by Data Science team, as it **reads from other repo's outputs**

1. The location of the local "data/" folder is a parameter in the code determined by "base path". If this folder wants to be located in a sharepoint or any other location, it is enough with adapting "base path" parameter in the code to the desired one

4 Front end - ERD

Entity Relationship Diagram (ERD) for Front-end pipeline input tables

NON-EXHAUSTIVE: ONLY MAJOR COMPONENTS DETAILED-OUT



Contents

Overview

1. Corridor model
2. Remote sensing
3. Risk model
4. Front-end

Data Structure

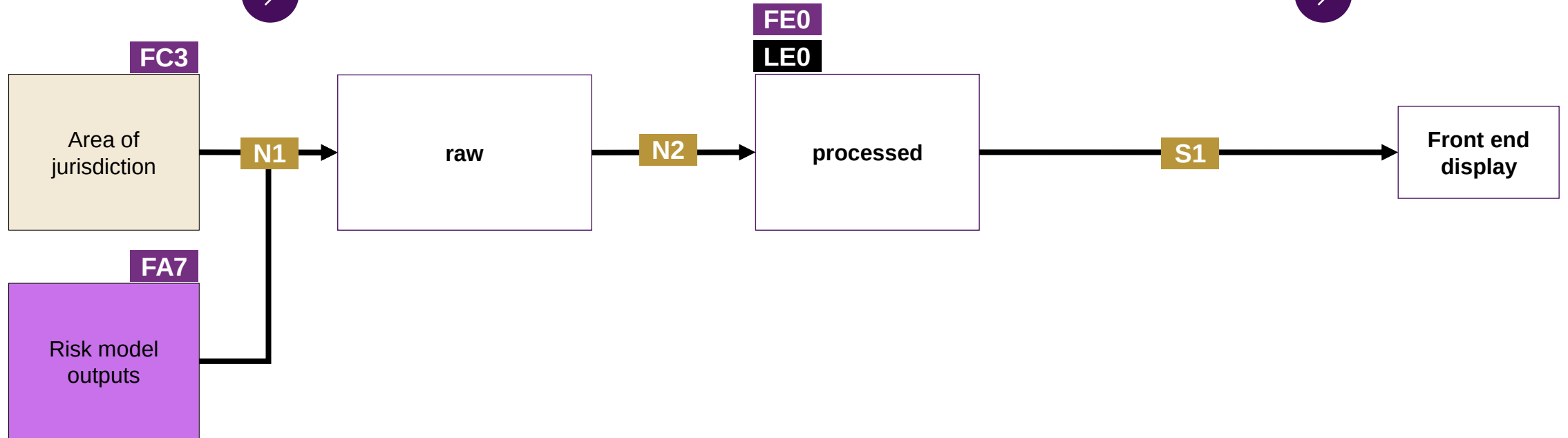
– Code repository

4 Front end - schema

Data
Extraction

Advanced Analytical Tool

Outputs



Notebooks (N)

N1: 01_process_raw_data.ipynb
N2: 02_pregenerate_maps.ipynb
S1.2: app.py

Data Folders (F)

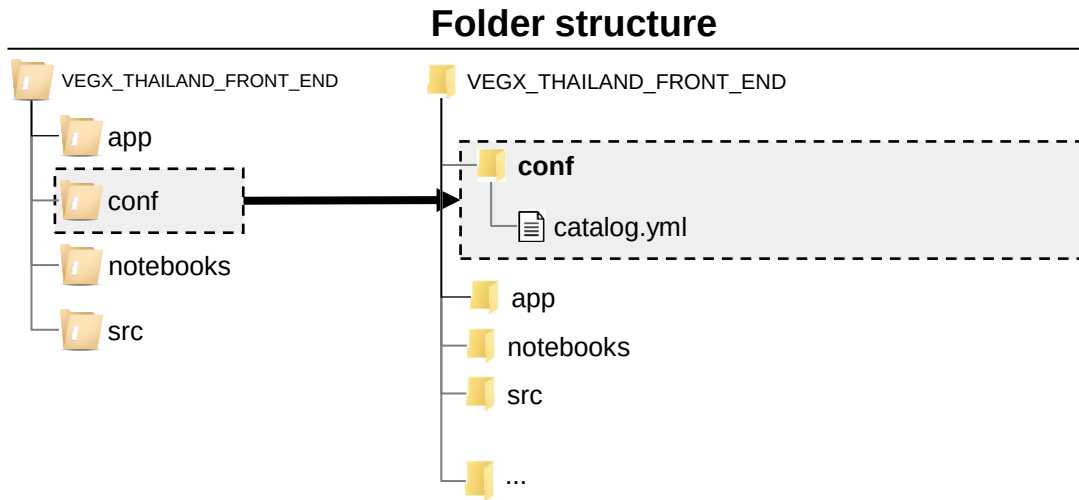
FC3: corridor_model.F3_modelresults
FA7: risk_modeling.F7_front_end_inputs
FE0: front_end_visualization.F2_processed
LE0: app/assets

Legend

 Risk model pipeline output
 Corridor mapping pipeline output
 notebooks
 Data path ID
 Local path

4 Front end model – conf/

conf/ - Catalog of files and parameters



> Content

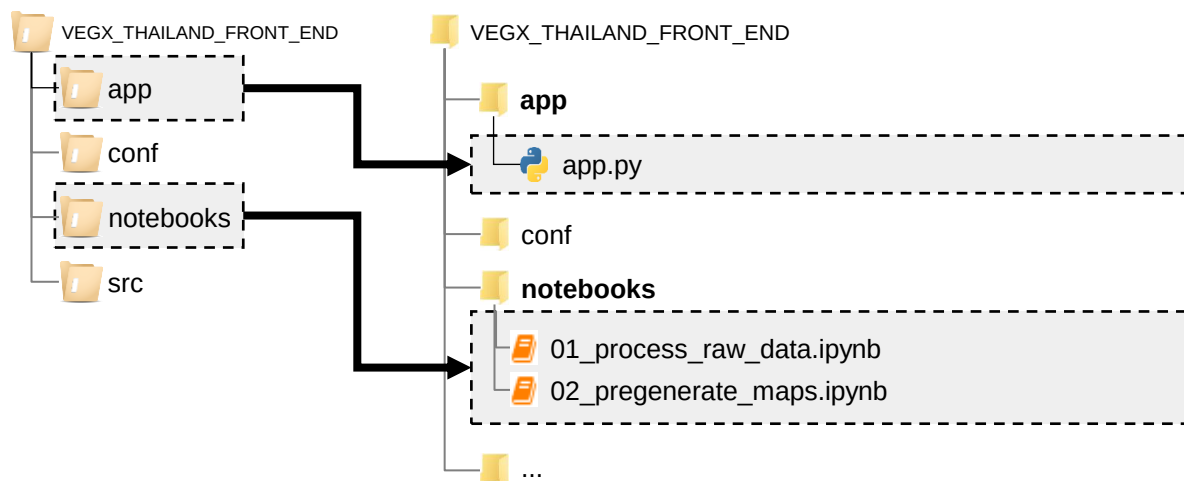
conf

- **catalog.yml**: contains the general files of the project.

4 Front end notebooks

notebooks/ and app/ - Examples and code repository

Folder structure



Content (read.me [link](#))

app/

- **app.py**: run dash app creation

notebooks/

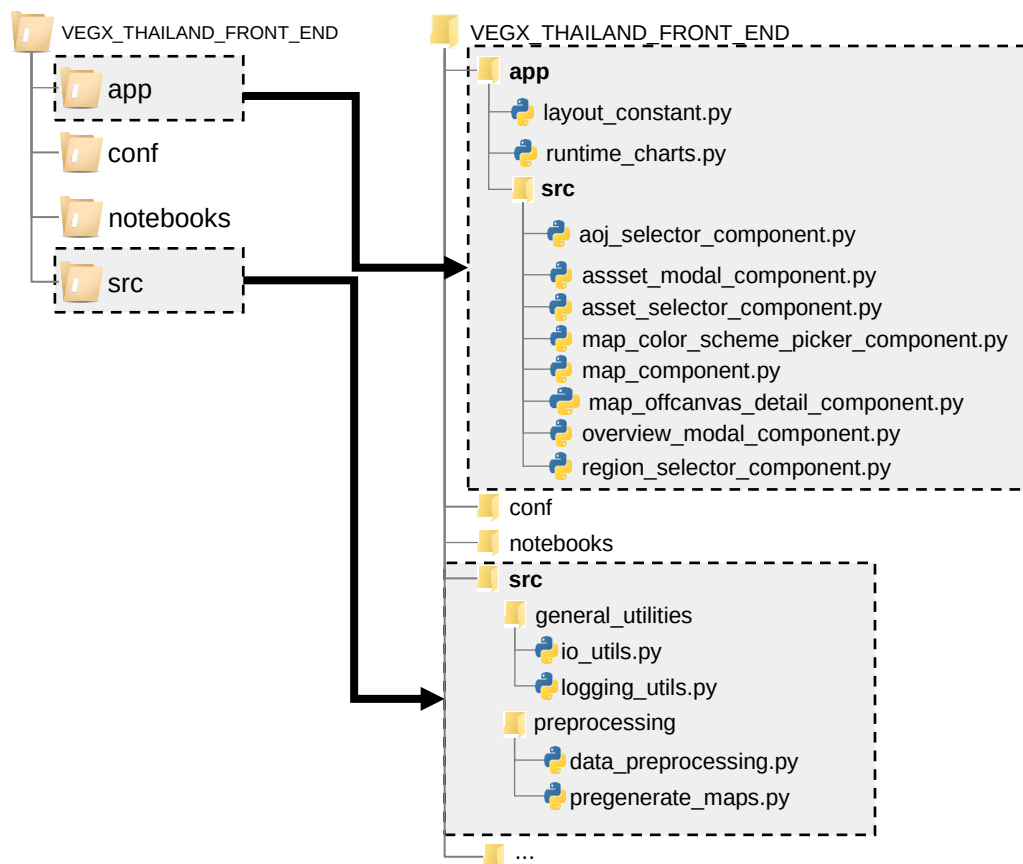
- **01_process_raw_data.ipynb**: stores data in
- **02_pregenerate_maps.ipynb**: generates maps to be displayed on dash app

4 Front end source scripts

src/ and app/ - Examples and code repository

The dash app is generally organized into components where a short script controls the generation and callback behavior of individual elements in the web app

Folder structure



Content

app/

- **layout_constant.py**: update any static charts or images
- **runtime_charts.py**: create charts at run time while viewing the map

app/src/

- **aoj_selector_component.py**: drop down selector for area of jurisdiction
- **asset_modal_component.py**: modal window with corridor specific views
- **asset_selector_component.py**: drop down selector for corridors
- **map_color_scheme_picker_component.py**: drop down selector for the color scheme used on map visualizations
- **map_component.py**: component that loads the html map files
- **overview_modal_component.py**: modal window with overall views
- **region_selector_component.py**: drop down selector for feeders
- **veg_content.py**: supplies dynamic content in modal windows

src/general_utilities

- **io_utils.py**: functions to import and save data files
- **logging_utils.py**: logging functions used throughout repository pipeline

src/preprocessing

- **data_preprocessing.py**: process input data from other pipelines
- **Pregenerate_maps.py**: generate html maps before running app